

PROJECT WORK

Bayesian Optimisation with Differentiable Simulated Priors for Particle Accelerator Tuning

Author:

Hiba Sikander

Supervisor: Jan Kaiser, M.Sc.

Jannis Lübsen, M.Sc.

Examiner: Prof. Dr.-Ing. Annika Eichler

March 1, 2026

Project Work

for Ms. Hiba Sikander

PA 25 - 08

Student ID: 610416

Study Course: M.Sc.
Data Science

Title:**Bayesian Optimisation with Differentiable Simulated Priors for Particle Accelerator Tuning****Project Description:**

The tuning of particle accelerators is a time-consuming and difficult task that continues to be performed manually in many cases, reducing the scientific output of some of the most advanced but also costly research facilities in the world. In the field of autonomous accelerator tuning, researchers aim to develop methods assisting human operators, thereby reduce tuning times and extend the experimental capabilities of accelerators. One of the most promising approaches to solve the autonomous accelerator tuning problem is the use of Bayesian Optimisation (BO), which is already successfully used in operations [1].

However, the most interesting accelerator tuning tasks are characterised by their high dimensionality. Despite being more capable at dealing with many dimensions than other algorithms, BO struggles to solve optimisations problems with more than a few dozen dimensions. To solve this challenge, researchers have proposed to leverage domain knowledge by replacing the commonly used but uninformative zero mean prior with a model of the system under optimisation. So far neural networks have been used for this purpose as they are differentiable and fast to compute. [2] While this work has shown the advantages of using an informed neural network prior, neural networks come with the disadvantage that they require large amounts of data to be trained, and that outside of the training data domain, their behaviour is ill-defined. In recent developments, high-speed differentiable simulators such as Cheetah [3] have been proposed as an alternative for use as a prior in BO, with the key advantage that they do not require any training data, and are well-behaved throughout. [3]

The goal of this research project is to take the BO with Cheetah prior approach presented on a very simple example in [3], and implement it on the closer-to-real world as well as more complex transverse beam parameter tuning task at the ARES accelerator, which has already been extensively studied with other autonomous tuning methods. The developed implementation should then be evaluated according to previous evaluations, to establish how this improved variation of BO compares to other methods and whether extension to even more complex accelerator tuning tasks is a promising avenue for future research.

Tasks:

1. Implement BO with Cheetah prior for tuning the transverse beam parameters on a simulation of the ARES Experimental Area.
2. Evaluate the BO with Cheetah prior implementation in accordance with the evaluation in [4].
3. Evaluate timing and speed of BO with an uninformed prior vs. BO with a Cheetah informed prior.
4. Study performance under model uncertainties.
5. **(Optional depending on accelerator availability)** Evaluate the developed implementation on the real ARES accelerator in accordance with [4].
6. **(Optional if time allows)** Setup a general implementation of BO with Cheetah.

References:

- [1] R. Roussel et al., “Bayesian optimization algorithms for accelerator physics”, *Physical Review Accelerators and Beams*, vol. 27, 2024.
- [2] T. Boltz et al., “Leveraging prior mean modules for faster bayesian optimization of particle accelerators”, *Scientific Reports*, vol. 15, 2025.
- [3] J. Kaiser, C. Xu, A. Eichler, and A. Santamaria Garcia, “Bridging the gap between machine learning and particle accelerator physics with high-speed, differential simulations”, *Physical Review Accelerators and Beams*, vol. 27, 2024.
- [4] J. Kaiser et al., “Reinforcement learning-trained optimisers and bayesian optimization for online particle accelerator tuning”, vol. 14, 2024.

Supervisor: Jan Kaiser, M.Sc.

Jannis Lübsen, M.Sc.

Examiner: 1. Prof. Dr.-Ing. Annika Eichler

Deutscher Titel:

Start Date: 17.11.2025

Due Date: 02.03.2026

March 1, 2026, Prof. Dr.-Ing. Annika Eichler

Hereby I declare that I produced the present work myself only with the help of the indicated aids and sources.

Hamburg, March 1, 2026

Hiba Sikander

Abstract

Tuning particle accelerators is an expensive and time consuming challenge that causes limitations in the findings of advanced research facilities. Bayesian Optimization (BO) has shown potential to be a promising approach for autonomous tuning of accelerators, yet its effectiveness diminishes for higher dimensional problems. Recent studies have proposed replacing the zero mean prior in BO with physics informed prior models to leverage physics domain knowledge. The Cheetah framework, a high-speed differentiable simulator, was demonstrated as a physics-informed BO prior on a simple two dimensional FODO lattice. This project work extends that approach to a real world five dimensional problem, which is the transverse beam parameter optimization at ARES Experimental Area involving three quadrupole magnets and two corrector magnets. This project implements a framework that incorporates Cheetah as the Gaussian Process mean function into the Xopt optimization toolset with trainable misalignment parameters that enable the prior to adjust when the simulator does not accurately represent the real system. Four optimization approaches (BO matched prior, BO mismatched prior, standard BO and Nelder-Mead) are evaluated. The results show that both physics-informed BO variants consistently achieve better performance in fewer iterations than both baseline methods, with the matched prior yielding the best overall performance across all evaluated metrics and BO with physics-informed prior remains effective even under prior-model mismatch. By significantly reducing the time and effort required for beam tuning, this work enables particle accelerator facilities to dedicate more time to experiments, pursue a wider range of research objectives and ultimately enhance their contribution to more scientific progress.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Extension From Simple FODO To Complex ARES Problem	2
1.3	Contribution	4
2	Theory	5
2.1	Gaussian Process	5
2.2	Bayesian Optimization	5
2.3	From Uninformed to Physics-Informed Prior	7
2.4	Cheetah-Based Differential Simulation Library	8
2.5	The ARES Accelerator	9
3	Implementation	11
3.1	Initial Setup	11
3.2	Physics-Informed Prior Mean Module	13
3.3	Optimization Runner	15
3.4	Evaluation Metrics	18
4	Result and Discussion	19
4.1	Final Results	19
4.2	Discussion	23
5	Conclusion and Outlook	25
6	Future Work	26

1 Introduction

1.1 Motivation

Particle accelerators are one of the most advanced scientific devices ever developed to perform innovative scientific experiments. The Deutsches Elektronen-Synchrotron (DESY) [1] located in Hamburg hosts multiple leading accelerator facilities such as European X-ray Free Electron Laser (XFEL). These facilities accelerate charged particles, that enable researchers to study atomic level processes and structures with great precision. A crucial challenge for particle accelerators is the tuning process, which involves adjustment of many magnetic components to produce desired beam properties including position, size and divergence. As a specialized test facility for accelerators research and development, Accelerator Research Experiment at SINBAD (ARES), which is a linear accelerator at DESY, allows the evaluation of innovative tuning techniques prior to deployment on even larger facilities.

Beam tuning is essentially an optimization task. In order to find exact configurations that minimize the deviation between the actual beam properties and the desired beam, we optimize the adjustable parameters (like magnet strengths and corrector/steering angles). However, the optimization is not that simple and perhaps contains a number of challenges:

1. **Expensive Evaluation:** It is impractical to conduct an extensive search since measurement of each beam property takes time and resources.
2. **Uncertainty:** There are a number of systematic errors and noises that might affect beam measurement.
3. **Black-box Nature:** The complex relationship between input parameters and beam characteristics cannot be directly observed analytically during operation.
4. **Unknown / Partial Observability:** Important parameters like characteristics of incoming beam and magnet misalignments are frequently either unknown or partially observed.

A traditional approach to accelerator tuning could be through skilled professionals who manually adjust parameters using their knowledge and experience, which is not really an efficient method. This method significantly relies on individual skill, takes a lot of time, is expensive and does not scale well to more complicated accelerator systems. Therefore, automated optimization methods are necessary for effective accelerator performance. Several optimization algorithms have been applied to particle accelerator tuning problems with their own unique advantages and disadvantages. Nelder-Mead Simplex (NM) is a gradient-free optimization technique and one of the approaches that is widely used because of its simplicity and robustness [2]. However, NM is best suited for convex optimization landscapes and tends to struggle in higher-dimensional search spaces. Another approach is the Reinforcement Learning (RL) technique that has shown remarkable performance [3]. RL agents can learn to tune accelerators in very few steps during deployment by training

neural network policies in simulation. However, RL requires extensive training data as well as very careful reward function design. The trained policies could not generalize well to conditions outside of their training distribution. The approach of our interest for this project work is Bayesian Optimization (BO), which finds a middle ground among these approaches. BO creates a probabilistic surrogate model of the objective function, which is usually a Gaussian Process, that provides estimates of uncertainty as well as predictions. This makes it possible for the assessment points to wisely balance between both exploration and exploitation. BO is a desirable option for accelerator tuning since it excels at solving expensive black-box optimization problems, thus BO is already used well with great success on accelerators [4–6] .

Another key insight motivating this project work is that, since the underlying physics of particle accelerators is well known, which means we can build reliable models based on physics knowledge. Thus, all thanks to modern simulation library like Cheetah [7] that takes advantage of this, making it possible to predict beam properties quickly and accurate enough to be used directly within an optimization loop. The standard BO uses uninformed zero mean prior function in the Gaussian process model. This implies that the algorithm must learn the whole function landscape from scratch and have to begin with no prior knowledge of the intended objective function values. On the other hand, if the physics-informed knowledge could be incorporated into the prior, this shall give advantage and optimization could already start with meaningful predictions that could focus its limited evaluation budget on identifying the differences between the model and the reality. The importance of physics-informed prior for accelerator optimization has also been shown in the recent work [8], where the neural network surrogate models trained on the simulation data can perform as effective prior mean functions, significantly speeding up the convergence. However, training neural network surrogates requires very large datasets from simulations.

Although, other simulators like Methodical Accelerator Design (MAD-X) [9] and ELEc-tron Generation ANd Tracking (elegant) [10] have been around for years and are capable of simulating beam dynamics, they do not compute the gradients directly and must rely on numerical gradient estimation methods such as finite differences, this makes them unsuitable for integration as a GP mean function, which would be highly inefficient as the surrogate model parameters must be updated at every optimization step. Cheetah being built on PyTorch, can compute gradients directly through automatic differentiation., enabling it to be embedded as the GP mean function with its parameters optimised jointly alongside the surrogate model.

1.2 Extension From Simple FODO To Complex ARES Problem

Cheetah has already been demonstrated as a physics-informed prior for BO on a simple and straightforward beam focusing FODO (Focus-Drift-Defocus-Drift) lattice [7]. A FODO cell is a fundamental building block of an accelerator lattice design, which is made up of alternating quadrupole magnets for focusing and defocusing that are separated by drift spaces.

Since this optimization problem was two dimensional, thus it only involved strengths of two quadrupole magnets (k^{Q1} and k^{Q2}) with the objective of minimising beam size at target location, defined as

$$mae = 0.5(|\sigma_x| + |\sigma_y|), \quad (1.1)$$

where σ_x and σ_y are the horizontal and vertical beam sizes, considering only beam size (focusing) without beam position (centering), unlike ARES problem, reflecting the simpler nature of FODO demonstration. Moreover, although there were two drift spaces (d_1 and d_2), they were constrained to have same lengths between the quadrupoles, therefore only a single trainable parameter was there, which was restricted to the interval $[0.2, 1.0]$ m and was included in the physics-informed prior mean function. The results showed that BO with Cheetah prior converged to the true minimum within 15 steps, indicating better sample efficiency than Nelder Mead Simplex and standard BO with zero prior. Furthermore, the method worked even with a mismatched prior, the algorithm learned the correct values during GP model fitting and converged to the true optimum when the drift lengths in the prior model (0.5m) was different from the ground truth (0.7m). While this validated the fundamental concept, the FODO problem is a simple example which does not depict the complex nature of actual accelerator tuning operations. The two dimensional search space is quite easy to explore and the model reality mismatch was restricted to a single geometric parameter (drift length) instead of the various other sources of uncertainty found in actual accelerators.

This project extends the FODO proof of concept to a significantly more complex and realistic problem which is transverse beam tuning at the ARES Experimental Area at DESY. The extension contain following key advancements:

1. **Higher Dimensionality:** In contrast to two dimensional FODO problem, ARES problem possess five dimensional search space including two steering/corrector magnet angles (θ^{CV} and θ^{CH}) and three quadrupole strengths (k^{Q1}, k^{Q2}, k^{Q3}). The volume of the search space is greatly increased by 2.5x in dimensionality, which makes it more difficult and raises the importance of informed prior.
2. **Realistic Lattice:** ARES Experimental Area is a real accelerator section loaded from an actual lattice description file unlike the synthetic FODO cell.
3. **Objective Nature:** For ARES problem, the objective function combines four beam parameters into a single metric, including horizontal and vertical sizes (σ_x, σ_y) as well as the positions (μ_x, μ_y). This makes simultaneous optimization of beam centering and focusing.
4. **Trainable Quadrupole Misalignments:** In real accelerators, quadrupole magnets are never perfectly aligned and cannot be measured directly. That is why, it is interesting to use quadrupole misalignments as the cause of model reality mismatch under this project work. The ARES prior includes 6 trainable misalignment parameters (horizontal and vertical for each quadrupoles Q1, Q2 and Q3) which are limited to physically plausible intervals of $\pm 0.5mm$, in contrast to the FODO demonstration, where only a single parameter (drift length) was learned.

1.3 Contribution

Thus the main contributions of this work are :

1. Demonstrating that Cheetah-informed prior approach is scalable from 2D synthetic lattice to 5D real world accelerator tuning problem.
2. Implementing physics-informed prior mean function for the ARES lattice being compatible with the Xopt Bayesian optimization framework
3. Extending prior with six trainable misalignment parameters, enabling the physics model to adapt to real accelerator conditions during optimization
4. Containing a comprehensive comparison between standard BO, Nealder-Mead and physic-informed BO (matched and mismatched prior) across multiple trials.

2 Theory

In this chapter, the theoretical foundations of this project work are developed. We begin with Gaussian processes as probabilistic function approximators, present Bayesian optimization as a framework for sample efficient black-box optimization and then look over the Cheetah simulator and the ARES accelerator where the work is used and how physics knowledge may be integrated through the prior mean function.

2.1 Gaussian Process

Gaussian process (GP) offers a robust framework for regression which allows uncertainty quantification along with predictions. GPs are well suited for modeling unknown objectives in optimization problems [4] because they define probability distributions directly over functions, unlike parametric models that learn a predefined set of weights. A Gaussian process is fully specified by its mean function $m(x)$ and covariance function (kernel) $k(x, x')$, thus can be written as

$$f(x) \sim \mathcal{GP}(m(x), k(x, x')), \quad (2.1)$$

which states that a function $f(x)$ is drawn from GP where the mean function $m(x)$ encodes previous expectations regarding the behavior of the function and the kernel $k(x, x')$ carries assumptions regarding smoothness and correlation between points. The kernel selection determines what kind of functions the GP considers probable. Because of its ability in modeling smooth, physically realistic functions without assuming infinite differentiability, Matérn-5/2 kernel is widely used in Bayesian Optimization [4]. The GP uses the Bayes's rule,

$$p(f|X, y) \propto p(y|f, X)p(f|X), \quad (2.2)$$

to update from prior to posterior when we observe data. $p(f|X, y)$ is the posterior (updated GP), $p(f|X)$ is prior (GP) and $p(y|f, X)$ is the likelihood of observed data. The posterior mean remains Gaussian, with mean and variance that reflects both our prior assumption and observed evidence. One of the important characteristics is that posterior uncertainty is low in areas with dense observations and high in areas with sparse data, this natural uncertainty quantification is what makes GPs useful for guiding optimization.

2.2 Bayesian Optimization

Bayesian Optimization (BO) is a sequential method for determining the optimal value of expensive black-box functions [4]. It still works well when the gradient information is not available and each function evaluation is expensive. Because of these characteristics, BO is ideally suited for accelerator tuning, where it has emerged as a state-of-the-art method [5, 6]. The goal is to solve,

$$x^* = \arg \min_{x \in \mathcal{X}} f(x), \quad (2.3)$$

x represents the control variables (like magnet settings), $\mathcal{X}$ is the search space and $f(x)$ is the quantity to be minimised (beam error in our case). BO aims to find x^* using as few function evaluation as possible. The key idea is to use a probabilistic surrogate model (usually a GP) to guide the search rather than evaluating the expensive function everywhere. As shown in Figure 2.1, the optimization process is iterative. We start with a modest initial set of observations, fit a GP model to the data and then choose the next point to evaluate using an acquisition function. Exploration (sampling when uncertainty is large) and exploitation (sampling where the predicted value is good) are balanced by the acquisition function. After evaluating the true objective at the selected point, we update the dataset and repeat until the evaluation budget is exhausted.

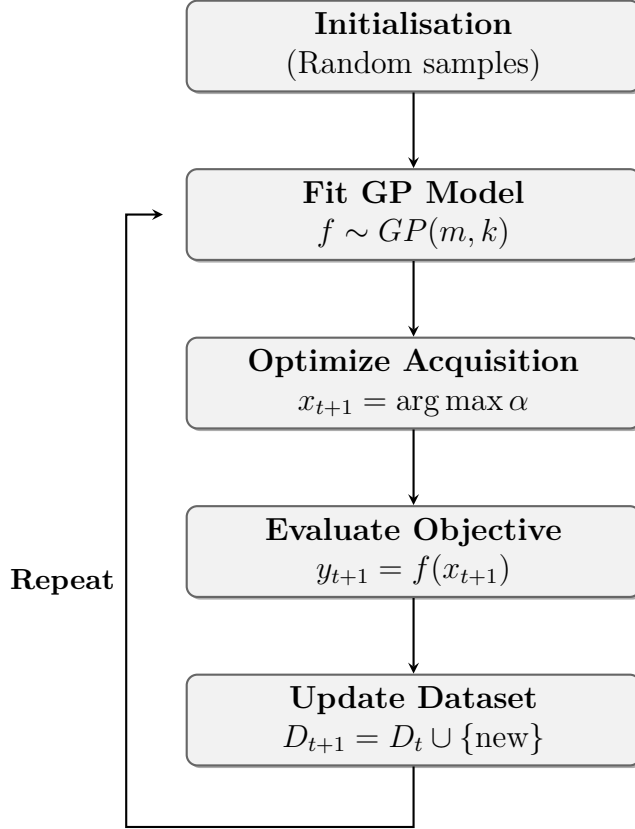


Figure 2.1: The Bayesian optimization loop.

Acquisition functions guide where to sample next by balancing two competing goals, where one is exploring uncertain regions while the other is exploiting areas that look promising. Two widely used acquisitions functions are Expected Improvement (EI) and Upper Confidence Bound (UCB).

- **Expected Improvement:** This acquisition function computes how much better do we expect a candidate point to be compared to our current best observation

$f(x^*)$, the expectation is taken over the GP’s predictive distribution. EI can be mathematically defined as

$$\alpha_{EI}(x) = \mathbb{E}[\max(0, f(x^*) - f(x))], \quad (2.4)$$

where $f(x^*)$ is the best (lowest) observed value so far. The expectation averages over what the GP thinks might happen at x and the max ignores cases where things get worse by ensuring that only improvements are counted. EI naturally trades off exploration and exploitation without any tuning parameter.

- **Upper Confidence Bound:** This acquisition function takes more direct approach by simply combining the predicted model and uncertainty. UCB can be mathematically defined as

$$\alpha_{UCB}(x) = -\mu(x) + \beta\sigma(x), \quad (2.5)$$

where $\mu(x)$ and $\sigma(x)$ are the GP posterior mean and standard deviation respectively, and parameter β controls the exploration-exploitation tradeoff.

This project work, uses the Upper Confidence Bound (UCB) acquisition function (Equation 2.5) with $\beta = 2.0$, following the approach used in the first demonstration over simple FODO problem [7] and common practice in accelerator optimization [4].

2.3 From Uninformed to Physics-Informed Prior

Before any data is observed, our belief about the objective is represented by the prior mean function $\mu(x)$. The standard practice is to use zero or constant mean, which assumes no prior knowledge. Physics models that approximate the system are often used in scientific applications and we have seen that incorporating this knowledge can significantly improve the performance [8]. To understand why, consider the GP posterior mean prediction at a new point. The GP learns to predict the difference between the prior mean and the actual function. These residuals are small and can be learned from fewer observations when the prior mean is accurate. Whereas when the prior is simply zero, the GP has to learn the entire function from scratch. This motivates using a physics-informed prior mean function, defined as

$$m(x) = f_{physics}(x; \phi), \quad (2.6)$$

with $f_{physics}$ being the physics model prediction, x being input to the optimization, which in our case are the five magnet settings required to be optimized (k_{Q1} , k_{Q2} , θ_{CV} , k_{Q3} , θ_{CH}) and ϕ represents parameters of physics model that are uncertain or unknown-like quadrupole misalignments, beam characteristics, drift lengths etc. In this project, it has been chosen to focus on quadrupole misalignments because they are a major source of model reality mismatch in ARES accelerator tuning. This benefits to improve sample efficiency and having meaningful predictions before observing data. In standard GP practice, which is

$$\omega^* = \arg \max_{\omega} \log p(y|X, \omega), \quad (2.7)$$

every time the GP model is fitted, it optimizes the kernel hyperparameters ω (length scales, signal variance, noise variance) by maximising the log marginal likelihood of observed data [11]. A key innovation in this project work is making the physics model parameters ϕ (in our case quadrupole misalignments: Δ_{xQ1} , Δ_{yQ1} , Δ_{xQ2} , Δ_{yQ2} , Δ_{xQ3} , Δ_{yQ3}) trainable alongside the standard GP hyperparameters, represented as

$$(\phi^*, \omega^*) = \arg \max_{\phi, \omega} \log p(y|X, \phi, \omega), \quad (2.8)$$

with y being the observed objective values, X being input points evaluated so far and ϕ being the physics model parameters. As more data is collected the learned ϕ gets closer to the true values so the physics model becomes more accurate eventually leading to better prediction and optimization performance. By optimizing $\log p(y|X, \phi, \omega)$, it measures how well the model explains the observed data. Note that ω is always optimized during standard GP fitting, while ϕ is only optimized when explicitly set as trainable. This approach allows the physics model to adapt better and gradually converge towards their true values, making the prior more accurate and speeding up the convergence by mitigating the mismatch between the model and reality.

2.4 Cheetah-Based Differential Simulation Library

Using physics simulator as a prior mean function requires both speed (so the simulator must be fast enough to call within each GP evaluation) and differentiability (in order to facilitate gradient-based hyperparameter optimization). Cheetah [7], was developed to satisfy these needs. Cheetah makes a conscious trade-off, in order to achieve speed, it gives up some fidelity. Its main objective is to give machine learning applications a differentiable beam dynamics code with improved speed over existing simulation codes. This ability of differentiation in Cheetah is important for our approach because when GP fits its hyperparameter by maximising marginal likelihood, gradients flow through the cheetah simulation to update the trainable misalignment parameter. Gradients of beam dynamics are often expensive to compute analytically or numerically [7] but Cheetah overcomes this by utilizing *PyTorch* automatic differentiation, which computes them numerically (to machine precision), while also being much faster. Additionally, Cheetah enables BO packages like *BoTorch* to optimize acquisition functions effectively. It provides two beam representations.

- **Parameter Beam:** assumes a Gaussian beam and represents it using a seven dimensional mean vector and covariance matrix, only these statistical moments are propagated through the lattice, enabling really fast simulation.
- **Particle Beam:** represents the beam as individual macro-particles, each described by seven-dimensional vector. This allows modeling of non-Gaussian distribution and bunch substructures, but at greater computational cost.

2.5 The ARES Accelerator

The Accelerator Research Experiment at SINBAD (ARES) facility at DESY serves as a testbed for advanced accelerator concepts and to test machine learning techniques on them. It is an S-band radio frequency linac with two independently powered travelling wave accelerating structures and a photoinjector. These structures can operate at energies upto 154 MeV. ARES's main goal is to create and investigate sub-femtosecond electron bunches at accelerated energy. For uses like radiation production by free electron lasers, the capacity to produce such brief bunches is highly desirable. ARES is also used for medical applications and for the study and development of accelerator components [3].

The project work focuses on the ARES experimental area. The beamline consists of three quadrupole magnets (Q1, Q2, Q3) and two corrector/steering magnets (CV for vertical steering and CH for horizontal steering), and a diagnostic screen where beam properties are measured. The magnets sequence of ARES Experimental Area, which is a subsection of ARES is depicted in figure 2.2 where two quadrupole magnets (Q1 and Q2) are followed by the vertical corrector magnet (CV), then the third quadrupole magnet (Q3) and finally the horizontal corrector magnet (CH) before reaching the screen. Each magnet is separated by a drift space ($d_1, d_2, d_3, d_4, d_5, d_6$) each having lengths within the range of 0.175 to 0.450 meters.

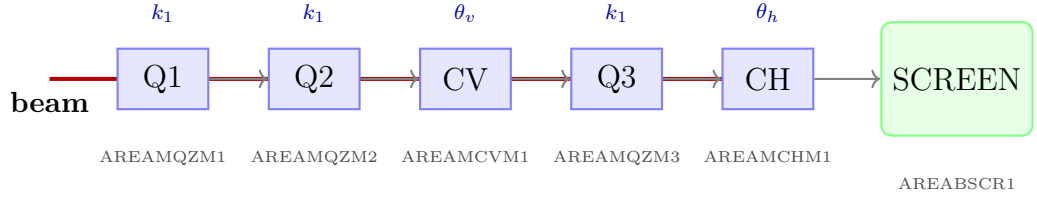


Figure 2.2: Schematic of the ARES Experimental Area showing the magnet sequence.

All magnets have polarity-switchable power supplies. It is possible to actuate the quadrupole magnets upto a 72 m^{-2} field strength. The steering/corrector magnets have a limit of 6.2 mrad. The camera's field of view is approximately 8 mm by 5 mm, with a pixel resolution of 2448 by 2040. The scintillating screen's effective resolution is about $20 \text{ }\mu\text{m}$ [3]. The beam tuning task involves optimizing the five magnets defined as,

$$\mathbf{u} = (k_{Q1}, k_{Q2}, \theta_{CV}, k_{Q3}, \theta_{CH}), \quad (2.9)$$

where $\mathbf{u} \in \mathbb{R}^5$. In order to produce a focused and centred beam at screen, The measured observables are beam's horizontal and vertical positions (μ_x and μ_y) and sizes (σ_x and σ_y). The objective function used throughout this work is

$$mae = \frac{1}{4}(|\mu_x| + \sigma_x + |\mu_y| + \sigma_y), \quad (2.10)$$

which is the mean absolute error which combines both centering and focusing of the beam into a single metric to minimise. Where σ_x and σ_y are strictly positive since they

are beam size. In a real world accelerator, the quadrupole magnets are never exactly aligned; the precise beam distribution entering the experimental area is not known. Even with the corrector magnets set to zero, a misaligned quadrupole with transverse offset (Δ_x, Δ_y) partially functions as steering element eventually diverting the beam. These misalignments are not directly measurable during operation. By treating the six misalignments parameters $(\Delta_{xQ1}, \Delta_{yQ1}, \Delta_{xQ2}, \Delta_{yQ2}, \Delta_{xQ3}, \Delta_{yQ3})$ as trainable quantities within the physics-informed prior, our approach addresses this issue too for the prior mismatched scenario and allows the model to adjust to actual accelerator settings using the optimization data.

3 Implementation

This chapter covers the implementation of Bayesian optimization with a Cheetah-based physics-informed prior for the transverse beam tuning at the ARES Experimental Area. Instead of being one large script, the implementation code is systematically divided into Python modules, each having their distinct functions:

- **bo_cheetah_prior_ares.py** includes the simulated ARES environment `ares_problem()` and the GPyTorch compatible prior mean class `AresPriorMean` with its trainable misalignment parameters. This is where most of the physics lives.
- **eval_ares.py** constructs the optimizers (Standard BO, BO with Cheetah prior and Nelder-Mead) along with the experimental scenarios (matched, mismatched and matched_prior_newtask) and loops through the optimization trials.
- **eval_ares_metrics.py** in accordance with the evaluation process [3], it calculates the performance metrics used to compare the techniques.
- **plot_ares_results.py** generates plots for the results.

The plotting and metrics modules are then connected by the fifth script **run_evaluation.py** to run altogether, it can also generate result tables in LaTeX format. Table 1 shows the summary of primary libraries used.

Libraries	Purpose	Version
Cheetah	PyTorch based differentiable beam tracking simulator.	0.8.0
GPyTorch	Gaussian process framework, base class for prior mean module	1.14.2
Xopt	BO loop, UCB acquisition Nelder-Mead baseline.	2.6.7
BoTorch	BO primitives used internally by Xopt	0.16.0

Table 1: Primary libraries/frameworks used.

3.1 Initial Setup

Before running any optimizers, we need something to optimize against. Defined in **bo_cheetah_prior_ares.py**, the method `ares_problem()` fulfills this purpose. A beam is propagated across a Cheetah model of the ARES experimental area using a set of magnet parameters, and the beam quality is then returned. In practice, every call to this function mimics what would happen if the user adjusted the magnet settings on the real machine and then looked at the diagnostic screen.

In this project, Cheetah’s `ParameterBeam` is used to describe the beam entering the ARES Experimental Area. `ParameterBeam` was used simply because it is faster, which enables it to run the experiments more quickly and iterate on the implementation with shorter turnaround times. Also note that Cheetah’s `ParticleBeam` could have also been just as well but with more computational cost. Since speed is not the most critical concern in this project, there is nothing holding us back from using either representation for this application. The ARES experimental area only involves linear optics, so both approaches get equivalent results. These values match the typical ARES operating conditions, with a beam energy of around 107 MeV and transverse sizes of 200 μm .

A JSON file describing the full ARES accelerator is used to load the lattice itself. The code extracts the sub-segment between AREASOLA1 (marker) and AREABSCR1 (screen) as we only require the experimental ares. Listing 1 demonstrates the loading of the lattice and the assignment of the five magnet parameters prior to tracking.

Listing 1: ARES lattice loading and magnet parameters configuration.

```

1
2 # Load ARES lattice
3 ares_segment = cheetah.Segment.from_lattice_json("
    ARESlatticeStage3v1_9.json")
4 ares_ea = ares_segment.subcell("AREASOLA1", "AREABSCR1")
5
6 # Set magnet parameters
7 ares_ea.AREAMQZM1.k1 = torch.tensor(input_param["q1"])
8 ares_ea.AREAMQZM2.k1 = torch.tensor(input_param["q2"])
9 ares_ea.AREAMCVM1.angle = torch.tensor(input_param["cv"])
10 ares_ea.AREAMQZM3.k1 = torch.tensor(input_param["q3"])
11 ares_ea.AREAMCHM1.angle = torch.tensor(input_param["ch"])
12
13 if misalignment_config is not None:
14     for magnet_name, (dx, dy) in misalignment_config.items():
15         magnet = getattr(ares_ea, magnet_name)
16         magnet.misalignment = torch.tensor([dx, dy], dtype=
            torch.float32
17     )
18 else:
19     ares_ea.AREAMQZM1.misalignment = torch.tensor([0.0,
20         0.0])
21     ares_ea.AREAMQZM2.misalignment = torch.tensor([0.0,
22         0.0])
23     ares_ea.AREAMQZM3.misalignment = torch.tensor([0.0,
24         0.0])
25
26 out_beam = ares_ea(incoming_beam)

```

For the three quadrupole strengths (k^{Q1} , k^{Q2} , k^{Q3}) and corrector angles (θ^{CV} and θ^{CH}), we have reduced the range from physical magnet’s $\pm 72\text{m}^{-2}$ to our simulation’s $\pm 30\text{m}^{-2}$, and from $\pm 6.2\text{mrad}$ to $\pm 6.0\text{mrad}$ respectively because the optimization consistently con-

verges well within the narrower range and exploring beyond it offers no practical benefit to the tuning process, since during practical operation, these ranges are usually never exceeded. The code iterates through the dictionary and applies each (Δ_x, Δ_y) pair to the corresponding quadrupoles when a misaligned configuration is provided. This enables us to model the realistic scenario in which magnets are not perfectly centred on the beam. To let the optimizer know how good or bad the result is, we need a single number. Following the previous FODO demonstration [7], for ARES Experimental Area, we use the mean absolute error as objective function, including each of the four beams observable, defined as

$$mae = 0.25(|\mu_x| + \sigma_x + |\mu_y| + \sigma_y), \quad (3.1)$$

with μ_x and μ_y being the beam positions (ideally, zero means beam is centred) and σ_x and σ_y being the beam sizes (the smaller the better), simultaneously centering and focusing the beam. The implementation is straight forward. The function also returns the individual beam parameters in addition to MAE so that they can be logged for later analysis.

3.2 Physics-Informed Prior Mean Module

The heart of this project work is the `AresPriorMean`, defined in `bo_cheetah_prior_ares.py`. Its role is to turn the Cheetah simulation into a format that GPyTorch can use as the mean function of the Gaussian Process. Instead of using GP prior (typically with zero mean assumption), it begins with the best estimates from the physics model and just needs to learn the residual which is the difference between what the simulator predicts and what is actually observed.

`AresPriorMean` class specifically inherits from `gpytorch.means.Mean` and implements a `forward()` method. Everytime GPyTorch needs the prior mean at a set of input points, it calls this method. The remaining GP components, such as kernel evaluation, posterior conditioning and acquisition function optimization, perform exactly like they would with any other mean function. Another important aspect is that Xopt builds the input tensor for the GP by sorting variables in alphabetical order, which means that since our magnets variables are named `q1`, `q2`, `cv`, `q3` and `ch`, column 0 of the tensor would always be `ch`, not `q1` as one might expect from the beamline order. In order to avoid silently feeding the wrong values to incorrect magnets, the class defines explicit index constants (Listing 2).

Listing 2: Explicit indexing to ensure correct mapping in Xopt input tensor.

```

1
2 # Load ARES lattice
3 class AresPriorMean(Mean):
4     # Considering alphabetical order
5     VAR_ORDER = ['ch', 'cv', 'q1', 'q2', 'q3']
6     IDX_CH = 0
7     IDX_CV = 1
8     IDX_Q1 = 2
9     IDX_Q2 = 3

```

As discussed earlier, in real accelerators, the quadrupole magnets are never perfectly aligned. A magnet that is even with a fraction of a millimeter, deflects the beam in a way that would not be predicted by a perfectly aligned simulation, partially acting as a steering factor. In order to handle this, the prior module treats the quadrupole misalignments as learnable parameters (Δ_{xQ1} , Δ_{yQ1} , Δ_{xQ2} , Δ_{yQ2} , Δ_{xQ3} , Δ_{yQ3}). Each of the three quadrupoles has horizontal and vertical misalignment values, making a total of six values. In order to do online system identification, GPyTorch’s marginal likelihood maximization adjusts these values in addition to the standard GP hyperparameters as the optimizer collects data. Three coordinated layers are used in GPyTorch to register each misalignment parameter, these three layers are summarized in table 2. On top of this, `@property` accessors offer a convenient interface, reading a misalignment property applies the forward transform (raw to constrained) and when it is written, the inverse transform is applied. This three layer pattern is repeated identically for each of the six parameters. The constructor is fairly lengthy because of this six times repeated pattern, although a loop based factory would have been cleaner, but we go with the explicit version since it has the advantage of being easy to read and to debug.

Component	GPyTorch API	What it does
Raw Parameter	<code>register_parameter()</code>	A raw unconstrained <code>nn.Parameter</code> is created at first, which is what PyTorch’s optimizer updates directly.
Interval Constraint	<code>register_constraint()</code>	Secondly, an interval constraint is used that maps this raw value through a sigmoid based transformation into the physically meaningful range of $\pm 0.5mm$, this serves as the bound guaranteeing that the learnt offset can never be greater than what is practical for quadrupole misalignments.
Smoothed box prior	<code>register_prior()</code>	Third, a <code>SmoothBoxPrior</code> is registered across the confined space, adding a soft regularization on top of the hard constraint. This helps the optimizer avoid getting stuck at the boundaries by gently penalizing values near the edges while assigning a fairly uniform probability throughout the interval.

Table 2: Three layered pattern to register misalignments as trainable parameters.

When GPyTorch calls `forward()`, the method receives a tensor `X` whose last dimension contains 5 columns (the five magnets, in alphabetical order), the steps include:

1. Using the index constants from Listing 2, slice out each magnet parameter.

2. Assign them to corresponding components of Cheetah lattice.
3. Apply the current misalignment values to the three quadrupoles by stacking them.
4. Run the beam through the lattice
5. Return the MAE (Equation 3.1).

Since all of these operations (the beam tracking, the absolute values, and the summation) are native PyTorch operations, the full forward pass is differentiable. Gradients could flow backward through the simulator and into the six misalignment parameters, which is what allows the joint optimization of both misalignment values as well as usual GP hyperparameters.

3.3 Optimization Runner

After setting up the environment and the prior setup, the remaining part is the runner that actually executes the experiments. This is handled by an `eval_ares.py` file that takes arguments for the experimental scenarios (matched, mismatched, matched_prior_newtask), the number of trials, the evaluation budget and the type of optimizer (BO, BO_prior, NM) where BO is the standard Bayesian Optimization, BO_prior is Bayesian Optimization with physics prior and NM is Nelder-Mead. Xopt defines the optimization problem declaratively using a YAML based specification known as VOCS (Variable and Objective Configuration Specification). Listing 3 shows the five variables and the single objective. Where the unit for quadrupole magnet ranges are m^{-2} , for corrector magnets ranges are rad and for objective (mae) is $metre(m)$

Listing 3: VOCS configuration.

```

1 vocs_config = """
2     variables:
3         q1: [-30, 30]
4         q2: [-30, 30]
5         cv: [-0.006, 0.006]
6         q3: [-30, 30]
7         ch: [-0.006, 0.006]
8     objectives:
9         mae: minimize
10 """
11 vocs = VOCS.from_yaml(vocs_config)

```

To test the prior under various conditions, the runner supports three scenarios given below, they are also summarized in Table 4:

1. **Matched Scenario:** This is the most simple and ideal scenario where the beam and misalignments in the environment are exactly what the prior expects, the default values (including all misalignment values to be zero), thus the physics model

perfectly matches reality. This scenario exists mainly to verify that the implementation works correctly and to see how much the prior helps when it is perfectly accurate since there is no model reality gap at all. In practice, this situation never happens on a real machine.

2. **Mismatched Scenario:** In this scenario, a gap is introduced on purpose to create mismatch between the prior and the reality, a different beam (σ_y changed from $200\mu\text{m}$ to $100\mu\text{m}$), and non zero misalignment values over all three quadrupoles are used in the environment (Table 3). The prior, however, starts from the default beam and zero misalignments as it does not know about any of these changes. We tell Xopt to include the six misalignment parameters in the GP’s marginal likelihood optimization by setting `trainable_mean_keys=["mae"]` in the `StandardModelConstructor` so they can be learned from data as it is received. This is the scenario that tests whether online systems identification actually works.

Magnet	Δ_x (mm)	Δ_y (mm)
AREASMQZM1	0.000	+0.200
AREASMQZM2	+0.100	-0.300
AREASMQZM3	-0.100	+0.150

Table 3: Ground truth quadrupole misalignments.

3. **Matched_prior_newtask Scenario:** The simulated accelerator setup is the same as in the mismatched scenario (modified beam and non-zero misalignments), but this time the prior is initialised with the correct values which are the right beam parameters and the true misalignment values. So this way, we are handing the prior a perfectly adjusted physics model and asking how fast can the optimizer converge when it already knows the true system’s state, compared to having to learn it during the process.

For the evaluation in this project, the `matched_prior_newtask` serves as our actual matched case, since it uses the realistic accelerator setup having modified beam and misalignments rather than the ideal zero misalignment defaults. We compare four methods, Nelder-Mead, Standard BO (zero mean), BO with Cheetah prior (mismatched scenario) and BO with Cheetah prior (matched_prior_newtask scenario). All four approaches face the same simulated accelerator so that the comparison is fair, what differs between them is only the optimization strategy, and in case of prior informed methods, how much the prior knowledge is available from the start for each method.

Three optimizers are compared Listing 4 shows how the most complex one (BO with mismatched prior) is set up. the other two are the simpler subsets of the same pattern.

Listing 4: BO with Cheetah prior(mismatched setup).

```

1 # Create the prior mean module with default
2 prior_mean_module = bo_cheetah_prior.AresPriorMean(
3     incoming_beam=incoming_beam

```

```

4         )
5         # Initialize misalignments to zero, will be
           learned
6         prior_mean_module.q1_misalign_x = 0.0
7         prior_mean_module.q1_misalign_y = 0.0
8         prior_mean_module.q2_misalign_x = 0.0
9         prior_mean_module.q2_misalign_y = 0.0
10        prior_mean_module.q3_misalign_x = 0.0
11        prior_mean_module.q3_misalign_y = 0.0
12
13        #Inject into GP constructor with
           trainable_mean_keys
14        gp_constructor = StandardModelConstructor(
15            mean_modules={"mae": prior_mean_module},
16            trainable_mean_keys=["mae"],
17        )
18
19        # Create UCB generator with beta 2.0
20        generator = UpperConfidenceBoundGenerator(
21            beta=2.0, vocs=vocs, gp_constructor=
                gp_constructor
22        )

```

Scenario	Modified Beam	True Misalignments	Prior Misalignment	Trainable ϕ
Matched	—	—	—	—
Mismatched	✓	✓	0	✓
Matched prior (new task)	✓	✓	= true	—

Table 4: Three experimental scenario summary.

In order to have a fair comparison, every trial starts from the same initial points, which are

$$x_0 = (k^{Q1} = 10.0, k^{Q2} = -10.0, \theta^{CV} = 0.0, k^{Q3} = 10.0, \theta^{CH} = 0.0). \quad (3.2)$$

After this initial evaluation, the runner loops through the `xopt.step()` for a fixed number of iterations. The default in the code is 50 steps, but for the evaluation presented in this report, we used 100 steps to give each method more room to converge and to better observe the later stage behaviors of the optimizers. Each step involves fitting the GP to all the data collected so far, selecting the next point by optimizing the UCB acquisition function and testing that point on the simulated accelerator. In the case of Nelder-Mead, the same `step()` interface updates the simplex instead. For mismatched scenario when the prior’s misalignment parameters are trainable, fitting the GP also involves optimizing

over ϕ . As a result, each new data point gives the marginal likelihood optimizer more information about where the magnets are actually positioned and the prior gradually gets more accurate which is the online system identification already discussed above. After each trial completes, the script stores the full MAE history, the overall best, and the learned misalignment values (where applicable). With a fixed random seed, ten separate trials are conducted per configuration and the results are saved to CSV files for further analysis.

3.4 Evaluation Metrics

The evaluation code is located in `eval_ares_metrics.py`. For each metric, $i \in \{1, 2, \dots, 10\}$, where i denotes the trial index, $t \in \{0, 1, \dots, T\}$, where t is the step (iteration) index and T is the last step which is 100 in our experiments.

1. **Best MAE:** is the best (minimum) MAE reached across all steps within each trial, defined as

$$\text{Best MAE}^{(i)} = \min_{t \in \{0, 1, \dots, T\}} \text{MAE}_t^{(i)}. \quad (3.3)$$

2. **Final MAE:** is the MAE at the last step of each trial ($\text{MAE}_T^{(i)}$).
3. **RMSE:** is the root mean square across all iterations within each trial, capturing how quickly the optimizer improves and not just the final value, defined as

$$\text{RMSE}^{(i)} = \sqrt{\frac{1}{4(T+1)} \sum_{t=0}^T ((\mu_x^{(i,t)})^2 + (\mu_y^{(i,t)})^2 + (\sigma_x^{(i,t)})^2 + (\sigma_y^{(i,t)})^2)}. \quad (3.4)$$

4. **Improvement Best percentage(%):** is the improvement measured from initial to best mae, computed as $((\text{MAE}_0 - \text{MAE}_{\text{best}})/\text{MAE}_0) \times 100$.
5. **Improvement Final percentage(%):** is the improvement measured from initial to final mae, computed as $((\text{MAE}_0 - \text{MAE}_{\text{final}})/\text{MAE}_0) \times 100$.
6. **Steps to target:** shows how many evaluations does it take to drop below the threshold of $40 \mu\text{m}$ of MAE for the first time.
7. **Success rate:** is the percentage of trials that reach the $40 \mu\text{m}$ target within the budget.

Though all the metrics provide quite good insights, the final error (the best MAE achieved) is the most crucial one for the practical purposes, and the steps to target are also informative because it tells how quickly the optimizer gets the beam into an acceptable state. `plot_ares_results.py` produces plots and tables, styled to match in [7] and [3], which can then be seen later under Section 4.

4 Result and Discussion

4.1 Final Results

This section presents the results of comparing four optimization strategies on the ARES Experimental Area beam tuning problem including Nelder-Mead, standard BO, BO_prior (mismatched), and BO_prior (matched_prior_newtask). All experiments were run for 100 iterations over 10 independent trials.

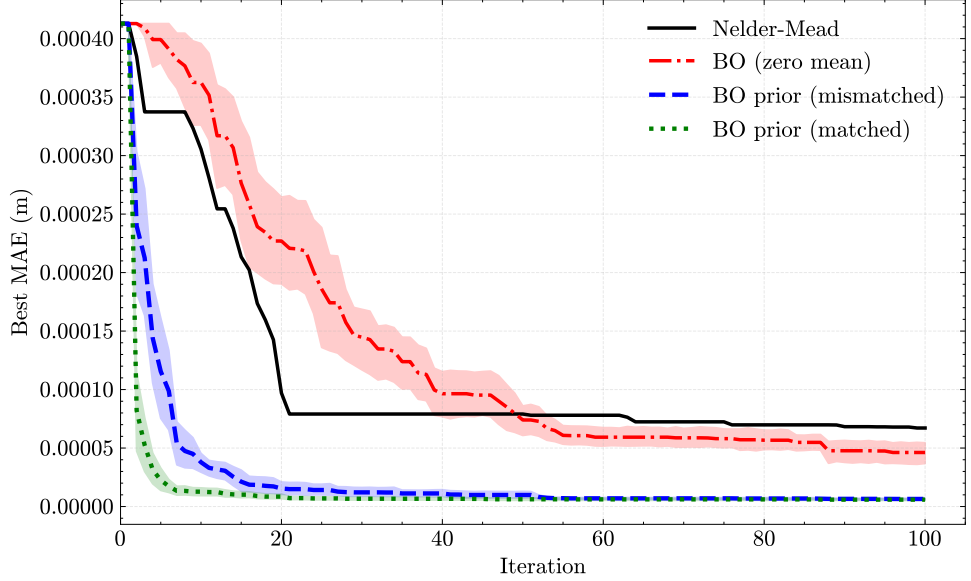


Figure 4.1: Convergence curve with cumulative minimum MAE over iterations.

The convergence curves plot in Figure 4.1 highlights the significantly faster convergence of the two physics-informed BO variants compared to the standard BO and Nelder-Mead baselines. The matched prior (green dotted line) drops sharply within the first five iterations, and flattens out by iteration 10 already. The mismatched prior (blue dashed line) is only slightly behind the matched case which makes sense because the GP needs some observations before the optimizer can start correcting the misalignment parameters, thus this case takes roughly 20 iterations to reach a comparable level to the matched case. The trajectory of standard BO with zero mean (red dashed-dot line) is somewhat more gradual. It does improve steadily, but even by iteration 60, it is still clearly above the physics informed methods. Additionally, the broad shaded region around indicates more uncertainty in BO standard case, reflecting the fact that without any guidance from prior, the initial random exploration phase can either go well or poorly, depending on where the points happen to land. Nelder-Mead (solid black line), in the early phase (around iterations 10-20), it actually decreases more quickly than standard BO, which is the characteristics of the simplex method to make large directed steps once it finds a downhill direction. However, it plateaus around $80 \mu\text{m}$ and then never goes lower than that. This shows that NM struggles in this five-dimensional space.

Table 5: Final Performance Metrics (100 iterations, 10 trials, target threshold for steps to target and success rate = 40 μm).

Optimizer	Best MAE (μm)		Final MAE (μm)		RMSE (μm)	
	Median	Mean	Median	Mean	Median	Mean
Nelder-Mead	67.13	67.13 $\pm$ 0	67.84	67.84 $\pm$ 0	229.20	229.20 $\pm$ 0
BO (zero mean)	45.88	46.21 $\pm$ 14.70	147.96	153.29 $\pm$ 66.66	844.64	822.19 $\pm$ 118.14
BO + prior (mismatch)	6.71	6.49 $\pm$ 1.11	40.54	41.71 $\pm$ 16.99	219.90	240.16 $\pm$ 65.58
BO + prior (matched)	5.75	5.89 $\pm$ 1.06	42.60	42.09 $\pm$ 16.90	177.57	173.11 $\pm$ 19.34

Optimizer	Improv. Best (%)		Improv. Final (%)		Steps to target		Success rate
	Median	Mean	Median	Mean	Median	Mean	
Nelder-Mead	83.74	83.74 $\pm$ 0	83.57	83.57 $\pm$ 0	—	—	0%
BO (zero mean)	88.89	88.81 $\pm$ 3.56	64.16	62.87 $\pm$ 16.15	82.5	82.5 $\pm$ 5.5	20%
BO + prior (mismatch)	98.37	98.43 $\pm$ 0.27	90.18	89.90 $\pm$ 4.12	10.5	10.8 $\pm$ 4.87	100%
BO + prior (matched)	98.61	98.57 $\pm$ 0.26	89.68	89.81 $\pm$ 4.09	4	3.6 $\pm$ 1.28	100%

As we can see from the final computed table, the final error (mae) for both physics informed BO variants (matched and mismatched) achieve a median of $6.22\mu\text{m}$ and $6.49\mu\text{m}$ respectively, which is better than Nelder-Mead($67.13\mu\text{m}$) and standard BO($45.88\mu\text{m}$). In practical terms, $6.22\mu\text{m}$ and $6.49\mu\text{m}$ are well below the $40\mu\text{m}$ threshold, so the optimizer has effectively pushed the beam to the best state that could be measured by an actual machine. Similarly, for the Improvement percentage, both the physics based approaches minimize the error by over 98%, whereas standard BO manages about 88.89% and Nelder-Mead about 83.74%. Even the standard deviations also shows worth noticing results, the physics informed results are reasonably consistent across trials, whereas standard BO has a spread of approximately ± 4 , which confirms the visual impression from the convergence curves plot that it is more sensitive to initial exploration phase. The success rate and steps to target columns shows quite impressive results as well. Setting the target of $40\mu\text{m}$ (which is a reasonable threshold set for a good enough beam quality), both physics informed BO variants reach it in every single trials making it a success rate of 100%, where the matched prior scenario gets there in just 4 steps and mismatched prior scenario gets there in 14 steps. The standard BO only succeeds in 2 out of 10 trials making a success rate of 20% and when it works, it requires a median of around 82 steps. Nelder-Mead never reaches the target within the 100 steps budget.

Methods that spend a lot of iterations at high MAE values before converging are penalized by the RMSE metric. The RMSE metric also shows how both variants of physics informed BO are better than the baseline methods. The matched prior BO has the lowest RMSE ($59.83\mu\text{m}$), reflecting the fact that it starts producing good predictions from the very first steps. The mismatched prior BO is slightly higher ($70.89\mu\text{m}$), since it spends a few early iterations adjusting its internal model. Nelder-Mead and standard BO both have RMSE values exceeding $140\mu\text{m}$ due to their long initial phase where they are still exploring.

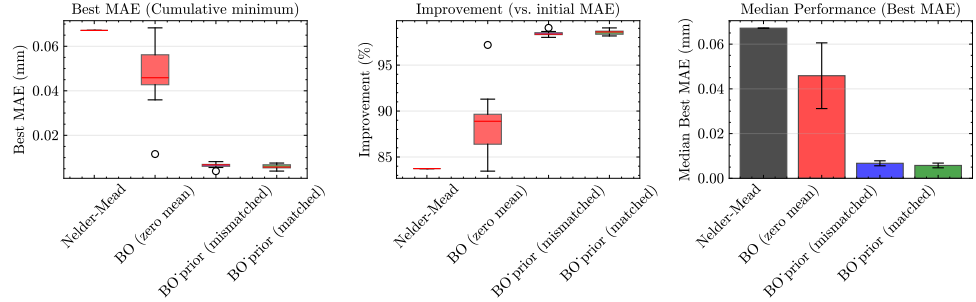


Figure 4.2: Box plot comparison across 10 trials. Left: final MAE (best achieved) for each method. Centre: percentage improvement from the initial beam state. Right: median final MAE with standard deviation error bars.

Figure 4.2 breaks down how each method performed on a per trial basis. In the left panel, Nelder-Mead collapses to a single line at $67\mu\text{m}$, indicating that it is getting stuck to the same local minima every time. Standard BO shows a wide box ranging from $25\mu\text{m}$ to $60\mu\text{m}$ with one lucky outlier near $10\mu\text{m}$, whereas both the physics informed BO variants sit tightly around $6\mu\text{m}$. The central panel confirms that the prior based BO methods cluster above 98% improvements, BO scatters between 84% and 97% whereas Nelder-Mead stays

below 84%. And finally the right panel’s median bars make the order of magnitude gap clearly visible. Figure 4.3 provides another perspective by plotting each trials’s final MAE against how many steps it took to converge.

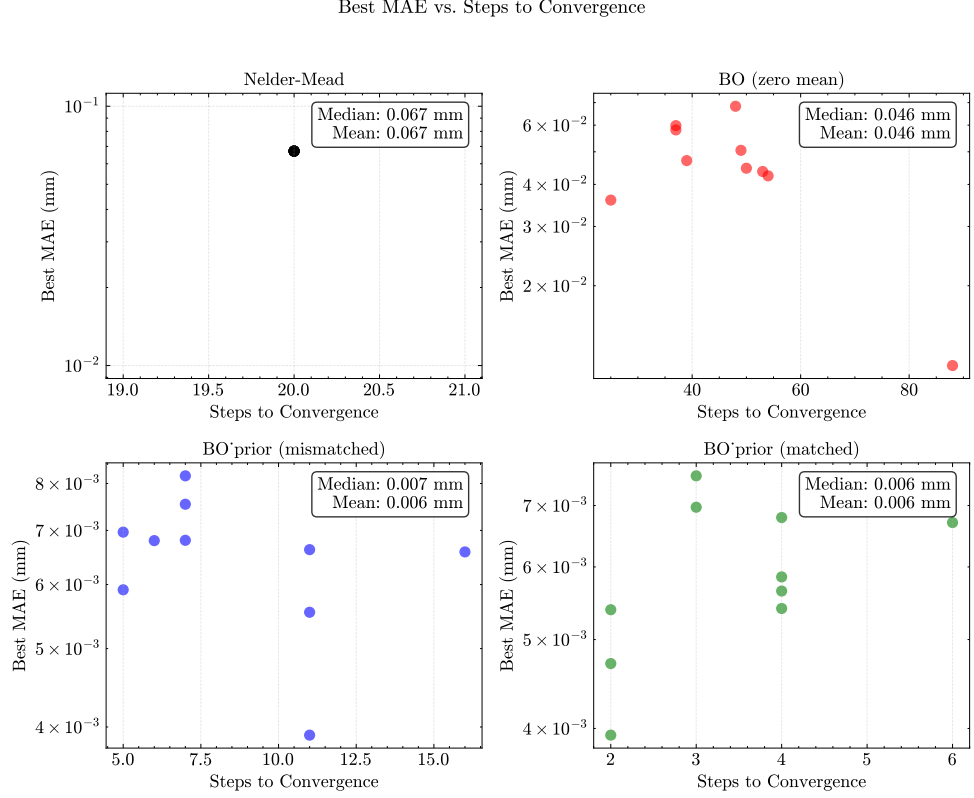


Figure 4.3: Final MAE versus steps to convergence for each trial. Fast convergence to a low error. Note the logarithmic y-axis.

The bottom left corner of each panel is where we want to be because this shows that this trial found a good solution quickly, while a dot on the top right means that this trial took a long time and still ended up with poor results. The Nelder-Mead panel has all its dots clustered top-right stalling near $67\mu\text{m}$ after about 94 steps (slow and poor), standard BO is scattered across the middle with no clear link between convergence speed and final quality, this suggests that it’s unpredictable, some trials get lucky, some don’t, so it mostly depends on where the early random samples land whereas both physics informed prior based methods, consistently reach errors at or below $10\mu\text{m}$. The matched prior scenario, most of them cluster on the left side, converging within 5-30 steps. Because the physics model is already accurate from the start. The y axis spans only $4\mu\text{m}$ to $7\mu\text{m}$, which means that every single trial lands on a great solution, few dots further to the right (around step 60 and 80) are the trials where the GP kept fiddling with tiny improvements (like going from $5.8\mu\text{m}$ to $5.3\mu\text{m}$), but the final answer is just as good as fast trials. And finally for BO prior mismatched scenarios, the blue dots are spread more widely on the x axis (from step 15 to 90) but this makes sense because some trials figure out the misalignments quickly while some take longer. But even here the important thing is that at y axis, even the slowest trials still end up between $5\mu\text{m}$ to $10\mu\text{m}$. The one dot near $10\mu\text{m}$ is likely a

trial where the learned misalignments most probably did not quite meet the ground truth, but even that is still way better than anything what BO or Nelder-Mead achieved.

Whether the mismatched prior can recover the true quadrupole sets solely from the optimization data is a crucial concern. Lets recall that the ground truth misalignments range from zero to 0.3 mm with both positive and negative offsets, while the prior starts with all six values set to zero. By the end of the optimization process, the learnt misalignments values for each of the ten trials closely match the ground truth, despite starting from zero, confirming that the GP’s fitting is really able to pick up and collect more data. The estimates tend to settle around iteration 15, which is actually roughly the same point where the mismatched prior catches up with the matched prior scenario in the convergence curve.

4.2 Discussion

All the results and findings portray a consistent picture that Cheetah-based physics prior delivers a significant improvement across every metric, not just in how good the final solution is, but also in how quickly and reliably the optimizer gets there. There are several points worth discussing:

1. The GP is given a warm start by the prior mean function, meaning that its predictions already approximate the shape of the true objective. This means that acquisition functions can make intelligent decisions from very first iterations, rather than going through several steps of random exploration. The coverage curve plot makes this especially clear that physics informed methods are already below $100\mu\text{m}$ by 5 iteration, but standard BO takes upto iteration 60 or later.
2. One might expect mismatched prior BO to perform significantly worse than matched one due to the completely incorrect misalignment values given to it. Whereas, the gap is quite small where the median final errors are almost identical, and the mismatched prior only requires approximately 10 extra steps to attain the $40\mu\text{m}$ target (14 vrs 4). This robustness is encouraged, as real world deployments often involve some degree of mismatch too.
3. Though standard BO does eventually find solutions between $30\mu\text{m}$ and $60\mu\text{m}$ with two out of ten trials falling below $40\mu\text{m}$, but a 20% success rate and a median convergence of 82.5 steps to target are not practical for accelerators where the beam is expensive and operators need reliable tuning within few minutes. Whereas the physics based approaches tackle both concerns.
4. The fact that Nelder-Mead converges to the same point in every trial (zero standard deviation on the final error) indicates a local minimum instead of a global one. In a 5 dimensional space with multiple local minima, this is not unexpected from a simplex based method. Thus, this local minimum ($67.13\mu\text{m}$) is not considered acceptable in practice as it exceeds the $40\mu\text{m}$ threshold.

5. Compared to the original FODO demonstration [7], which was a two dimensional problem with a single learnable parameter, this project work used a 5 dimensional landscape with 6 learnable parameters. Still the methods with physics informed prior converged well in approximately 14 steps (mismatched) or 4 steps (matched). This shows that the approach scales reasonably well with the problem dimensionality in this scenario.

5 Conclusion and Outlook

The central question behind this project work was to find out whether a differentiable physics simulator, used as the prior mean in Bayesian Optimization, can actually help with a realistic accelerator tuning problem (ARES Experimental Area) or does it only work on a toy example (FODO lattice)? Based on everything presented, including implementation, experiments and the results, it can confidently be said "yes".

The problem tackled in this project was the transverse beam tuning task at the ARES Experimental Area, the idea of Cheetah based prior demonstrated on a simple two dimensional FODO lattice [7] was scaled up to a five dimensional beam tuning task now under this project scope, including 5 tuneable magnets (three quadrupole magnets and two corrector magnets), six unknown quadrupole misalignment values that the prior had to learn during the optimization and a four components based objective that asks optimizer to focus and centre the beam simultaneously. The code was built around a module `AresPriorMean` that wraps the Cheetah differentiable simulator and plugs into the Xopt framework. The project contains a modular codebase architecture, with clean separation between the prior mean function, the optimization loop, evaluation and plotting. The approach was tested against three experimental conditions. In the matched scenario, the prior already knows the correct system state, which is the best-case scenario. In the mismatched case, the modified beam and wrong misalignment values were given to prior on purpose, forcing it to learn the correct values during optimization. And in the matched_prior_newtask scenario, the correct misalignment values were given upfront, which was the matched case for evaluation and comparison purposes under this project. Both Cheetah prior based BO were compared against two baseline techniques, the standard BO with an uninformed prior and Nelder-Mead.

The results speak for themselves. Both the physics informed BO variants (matched and mismatched) achieved a median MAE of $6.22\mu\text{m}$ and $6.49\mu\text{m}$ respectively, which is significantly higher than standard BO ($45.88\mu\text{m}$) and Nelder-Mead ($67.13\mu\text{m}$). The matched prior hit the $40\mu\text{m}$ target in a median 4 steps, the mismatched prior in 14, whereas standard BO required 82.5 steps on a rare occasions (20% success rate) and Nelder-Mead never reached the target. What makes the approach especially promising for real world use is its robustness. Even when starting with incorrect misalignments values and modified beam distribution (the mismatched prior in BO case), the prior still outperformed the uninformed zero mean prior (standard BO) and caught up to the perfectly matched prior scenario of BO very soon within 15-20 iterations. The trainable misalignment parameters successfully converged very close to the ground truth purely from optimization data, which means that the method does not just tune the beam but it also learns something about the machine's physical state along the way.

Ofcourse, the ARES Experimental Ares with its five tuneable parameters is still a relatively small problem compared to large-scale facilities like European XFEL and PETRA III, but this is a significant step forward from the two dimensional FODO lattice that was previously tested to evaluate cheetah's potential. The fact that the method scales gracefully to this level sends quite good signs for scaling further and handling more complex beamlines in future.

6 Future Work

This project work has the potential to be extended in several directions ranging from immediate next steps to longer term research questions. The most obvious next step now would be to test the whole implementation on the real ARES accelerator rather than in simulation. A real machine could bring things that a simulator cannot completely capture, like measurement noise, small differences between cheetah model and actual machine etc. Since Xopt already supports a real hardware interface, the only modifications would be to replace the simulated evaluation function with a live one. Another interesting step for the future work could be to convert the code into a Python package. The system is currently tailored to the ARES, with hardwired magnets names, lattice file, and misalignments. But the idea of picking a cheetah lattice, tuning, learning and running BO itself is not specific to any one beamline, so packaging it up with a cleaner interface would make it much easier for others to try on their own accelerators without having to dig through ARES specific scripts.

Currently, each time the optimizer runs, it starts fresh, but in real accelerators the machine does not change drastically between the tuning sessions, The learned misalignment values and GP hyperparameters could be used as warm start for the next, potentially speeding up the convergence even further. So it would be worth having a look into it more. Some other research direction that could push this project work further for future work could be tuneable parameters increments. The ARES problem only has 5 tuneable parameters that were the 5 magnets, but a real beamline can have dozens of them At such scale the GP gets expensive and it gets harder for acquisition function to search, so combining the cheetah prior with dimensionality reduction techniques would be worth exploring.

The current project uses objective function that combines beam position and size into a single MAE metric. In some operational scenarios, it might be more natural to treat these as separate objectives (centering the beam and focusing the beam) and use multi-objective BO instead of combining everything to a single MAE, so this could be another direction to experiment. Furthermore, on the modeling side, only the quadrupole misalignments were learned in this project, but there could also be other sources mismatched/unknown in the real machine, like incoming beam distribution may not be perfectly known, thus experiments can be conducted to learn other unknown parameters as well. Finally, the tests could also be conducted on more advanced beamlines with more complex objectives like the ones used in European XFEL and PETRA III.

References

- [1] Deutsches Elektronen-Synchrotron DESY. “Desy - homepage”, Accessed: Feb. 9, 2026. [Online]. Available: <https://desy.de/>
- [2] J. A. Nelder and R. Mead, “A simplex method for function minization”, *The Computer Journal*, vol. 7, no. 4, 1965.
- [3] J. Kaiser et al., “Reinforcement learning-trained optimisers and bayesian optimization for online particle accelerator tuning”, vol. 14, 2024.
- [4] R. Roussel et al., “Bayesian optimization algorithms for accelerator physics”, *Physical Review Accelerators and Beams*, vol. 27, 2024.
- [5] S. Jalas et al., “Bayesian optimization of a laser-plasma accelerator”, *Phys. Rev. Lett.*, vol. 126, p. 104801, 10 Mar. 2021. DOI: 10.1103/PhysRevLett.126.104801 [Online]. Available: <https://link.aps.org/doi/10.1103/PhysRevLett.126.104801>
- [6] J. Duris et al., “Bayesian optimization of a free-electron laser”, *Phys. Rev. Lett.*, vol. 124, p. 124801, 12 Mar. 2020. DOI: 10.1103/PhysRevLett.124.124801 [Online]. Available: <https://link.aps.org/doi/10.1103/PhysRevLett.124.124801>
- [7] J. Kaiser, C. Xu, A. Eichler, and A. Santamaria Garcia, “Bridging the gap between machine learning and particle accelerator physics with high-speed, differential simulations”, *Physical Review Accelerators and Beams*, vol. 27, 2024.
- [8] T. Boltz et al., “Leveraging prior mean modules for faster bayesian optimization of particle accelerators”, *Scientific Reports*, vol. 15, 2025.
- [9] L. Deniau, *Methodicalaccelerator design/mad-x: 5.09.01*, Dec. 2023. DOI: 10.5281/zenodo.10255261 [Online]. Available: <https://doi.org/10.5281/zenodo.10255261>
- [10] M. Borland, “Elegant: A flexible sdds-compliant code for accelerator simulation”, Advanced Photon Source, Tech. Rep. LS-287, Sep. 2000.
- [11] C. K. Williams and C. E. Rasmussen, *Gaussian processes for machine learning*. MIT press Cambridge, MA, 2006, vol. 2.